

UI/UX Design System & Responsive Spec

High-fidelity first; modern polish second • Poppins • mobile-first responsive onboarding

1. Design direction

The strongest submission is not a generic trendy redesign. Treat the reference app as the visual authority and add 2026-level polish through consistency: crisp type, controlled motion, responsive composition, accessible focus states, excellent empty/error/loading states and carefully tuned spacing.

Visual language

- Font: Poppins, as explicitly permitted by the assessment.
- Use the reference app's actual palette after screenshot sampling. Do not introduce an unrelated SaaS palette.
- Prefer a restrained surface system: background, elevated form surface, primary text, muted text, border, primary action, destructive/error and success.
- Use a single radius scale and shadow language. Avoid mixing glassmorphism, neumorphism and heavy gradients unless visible in the reference.
- Use subtle motion (150–250 ms) for focus, step transition, toast and button loading. Respect prefers-reduced-motion.

2. Responsive composition

Viewport	Layout behavior	Target quality
Mobile 320–479	Single-column, reference-like full-height onboarding, sticky-safe CTA if appropriate	No horizontal scroll; comfortable keyboard interaction.
Large mobile 480–767	Centered content with wider gutters	Preserve mobile visual identity.
Tablet 768–1023	Centered onboarding panel; optional decorative side area only if it does not alter parity	Avoid oversized empty space.
Desktop 1024+	Max-width form column with balanced artwork/background treatment	Should feel intentionally designed for web, not a stretched phone screen.

3. Component specification

Component	States to design	Quality details
Text input	default, hover, focus, filled, invalid, disabled	Persistent label or reference behavior; 44px+ hit height; clear error association.
Primary button	default, hover, pressed, focus, loading, disabled	Loading keeps width stable and blocks repeated submit.
Select / combobox	closed, open, selected, invalid, disabled	Keyboard navigation; searchable only when option count justifies it.
OTP	empty, focused, partial, complete, invalid, resend waiting	Auto-advance without trapping keyboard; paste full code support.
Progress	step N of 4, completed steps, back	Do not rely on color alone.
Toast/banner	error, success, info	Do not hide field-specific errors only inside a toast.

4. Screen-by-screen polish

- Landing: strong visual focal point, exact CTA hierarchy, optimized background image, no layout jump when fonts load.
- Terms: readable 60–75 character line length on desktop, visible heading hierarchy, back/continue actions, no tiny legal text.
- Signup: one clear task per step, short explanatory copy, consistent vertical rhythm, Enter behavior that never skips validation.
- OTP: show destination email, resend timer, editable/back affordance, pasted-code support, clear expired/incorrect states.
- Age: if user is below the permitted age, show an immediate respectful blocking message instead of allowing silent progression.
- Success: confirmation icon/animation can be modern, but keep it short and deterministic.

5. Accessibility requirements

- Use semantic form labels, fieldset/legend for grouped choices, and aria-describedby for errors.
- Visible keyboard focus with at least a 3:1 focus-indicator contrast against adjacent colors.
- Minimum practical touch target around 44×44 CSS px for important controls.
- Announce async errors/success via aria-live without repeatedly interrupting the user.
- Never use placeholder text as the only label.
- Test at 200% browser zoom and with reduced motion enabled.

6. UI anti-patterns to avoid

- Over-designed glass cards that reduce fidelity.
- Excessive gradients, animated blobs, 3D scenes or particles unrelated to the reference.
- Replacing the app's exact step order with a 'better' flow.
- Desktop-only hover interactions.
- Tiny info icons or tap targets; a public review of the reference app specifically mentions small info-icon targets as a usability issue, so enlarge interactive hit areas while preserving visual size.
- Using animation to hide slow rendering or validation bugs.